PROJECT INTEGRATION REPORT

Shopify-Zapier to Zoho CRM Dual-Store ASP.NET Core Pipeline

Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
CRM	Customer Relationship Management
JWT	JSON Web Token
HMAC	Hash-based Message Authentication Code
SHA	Secure Hash Algorithm
EF Core	Entity Framework Core
SQL	Structured Query Language
JSON	JavaScript Object Notation
URL	Uniform Resource Locator
TLS	Transport Layer Security
HTTPS	HyperText Transfer Protocol Secure
XML	eXtensible Markup Language
WSL	Windows Subsystem for Linux
DDL	Data Definition Language
REST	Representational State Transfer

1. Introduction

Integrating diverse e-commerce networks with internal enterprise customer relationship platforms demands secure, highly-performant, and traceable pipelines. In modern retail logistics, transaction information generated at the client end must be verified for authenticity and securely written into both structured analytical data repositories and operational logs before being synchronized with outbound business management suites like Zoho CRM.

In response to this architectural need, this project presents a "Dual Mode Storage Integration Pipeline" built on ASP.NET Core 10.0. The pipeline securely receives order data via ingress endpoints adapted for both Shopify webhook channels and Zapier customized CLI applications, processes the data through dual relational (PostgreSQL) and non-relational (MongoDB) databases, and synchronizes the validated information with Zoho CRM.

1.1 Motivation

E-commerce systems deal with high-frequency transactions that require immediate processing, audit trails, and synchronization. Standard setups suffer from limitations: systems optimized for fast webhook response often fail to maintain audit logs of raw payloads, leading to tracing failures when outbound integrations drop. Moreover, lacking robust ingress key signatures makes endpoints highly vulnerable to spoofed transactions.

Our core project objectives include:

- **E-Commerce Security:** Implementing SHA256 HMAC handshakes for Shopify webhooks and OAuth 2.0 JWT for Zapier integrations.
- **Auditability:** Creating a dual-store logging model (PostgreSQL for clean order structures, MongoDB for raw payload audits).
- **Interoperability:** Structuring a Zoho CRM client service with mock failover profiles to test endpoints end-to-end.

2. Literature Survey

Modern microservice design patterns heavily rely on message queues and API gateway patterns to establish reliable distributed pipelines (Richardson, 2018). In e-commerce, the decoupling of transaction collection from CRM synchronization is crucial to avoid bottlenecking checkout operations. Standard models highlight the use of asynchronous event processing (Newman, 2019).

Security frameworks for webhooks, specifically regarding cryptographic HMAC verification, are established standards for validating request headers (RFC 2104). The use of JWT Bearer tokens for third-party application authorization provides secure stateless client authentication without needing database locks for active sessions (RFC 7519).

On the database front, polyglot persistence (using SQL and NoSQL databases in tandem) is recognized as an optimal strategy (Sadalage & Fowler, 2012). Storing transactional records in relational systems ensures transactional safety, while storing original, unstructured JSON payloads in document databases preserves strict logs without forcing schema constraints on incoming payloads.

3. Problem Statement and Objectives

3.1 Problem Statement

In modern enterprise architectures, processing transactions across multiple external boundaries (like Shopify webhooks and Zapier customized platforms) introduces severe challenges regarding:

- **Ingress Security:** Protecting API endpoints from spoofed request payloads or injection threats.
- **System Reliability:** Syncing transactions to external CRMs (like Zoho CRM) without risking data loss when APIs go offline.
- **Observability:** Ensuring all request contents and original JSON strings are preserved for audit checks.

3.2 Project Objectives

The proposed architecture resolves these issues by implementing a secure, dual-store ASP.NET Core web api pipeline. Our specific goals are:

- **Security Ingress:** Provide constant-time HMAC validation and stateful JWT token issuance.
- **Data Persistence:** Use PostgreSQL for query-optimized order records and MongoDB for unstructured payload logs.
- **Outbound Sync:** Synchronize clean data records with Zoho CRM via OAuth2 token exchange with mock fallbacks.

4. Requirements

4.1 Functional Requirements

- **FR01 Token Auth:** Zapier client obtains JWT token using client credentials and sends it in Bearer scheme.
- **FR02 Webhook Security:** Shopify webhook endpoint computes and validates HMAC signature against configurations.
- **FR03 Polyglot Storage:** Write structured order database schema to PostgreSQL and raw json requests to MongoDB.
- **FR04 Zoho CRM Sync:** Format clean order objects and POST them to Zoho CRM Contacts endpoint.
- **FR05 Sandbox Testing:** Toggle mock configurations for Zoho CRM and databases to validate flows offline.

4.2 Non-Functional Requirements

- **NFR01 Performance:** Ingress API requests complete in < 500ms, and database writes execute asynchronously.
- **NFR02 Security:** Encrypt data-in-transit via HTTPS and implement secure, constant-time bytes comparisons.
- **NFR03 Maintainability:** Provide a containerized environment using multi-stage Dockerfiles and docker-compose.yml.

5. Methodology

5.1 Development Approach: Agile Methodology

The project adopted the Agile software development lifecycle, utilizing short iteration cycles. Each iteration focused on specific components of the pipeline (API layer, Database configuration, Outbound clients, and Containerization).

5.2 System Architecture Overview

The system is divided into three coordinated architectural layers:

- **1. API & Security:** Kestrel-hosted endpoints verifying Shopify webhook HMAC signatures and Zapier OAuth 2.0 JWT Bearer headers.
- **2. Data Storage:** Asynchronous polyglot persistence writing structured data using EF Core and raw audits using MongoDB C# Driver.
- **3. Outbound CRM Sync:** Zoho CRM HTTP client service with automatic token refresh flows and mock sandbox controls.

6. System Design and Schema

The backend is engineered around clean architectural patterns, isolating the controller routes from database repositories and external integration clients. The database schemas emphasize strict data contracts for orders, combined with loose document stores for request payloads.

6.1 Polyglot Storage Design

By storing structured values in PostgreSQL, we optimize queries for order status, sync status, and total sales volume. Concurrently, MongoDB logs all headers and the exact incoming JSON string. This prevents database lockups if the order schema changes on the client side, ensuring audit compliance.

7. Data Flow

The system exhibits two main execution pathways depending on the transaction origin:

7.1 Shopify Webhook Dataflow

- **A. Webhook POST:** Shopify dispatches order payload to /api/webhooks/shopify containing the HMAC signature header.
- **B. Verification:** Controller checks signature against configurations. Mismatched calls return 401 Unauthorized.
- **C. Logging & Storage:** Raw body logged to MongoDB; structured entity saved to PostgreSQL Orders table.
- **D. Outbound Sync:** ZohoCrmService dispatches order fields to Zoho. SyncStatus is updated to "Synced" or "Failed" in Postgres.

7.2 Zapier Dataflow

- **A. Token Issue:** Zapier requests JWT token from /api/auth/token using developer client credentials.
- **B. Authenticated Request:** Zapier calls /api/orders sending the JWT in Authorization: Bearer header.
- **C. Execution:** Controller authorizes request, logs raw payload to Mongo, writes to PostgreSQL, and triggers Zoho sync.

8. Database and Table Schema

8.1 PostgreSQL Orders Schema (DDL)

The structured relational table "orders" stores the records generated by the pipeline. An explicit DDL script compiles the database schema:

```
CREATE TABLE orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id VARCHAR(100) NOT NULL UNIQUE,
    customer_email VARCHAR(255) NOT NULL,
    total_amount NUMERIC(12, 2) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    sync_status VARCHAR(50) NOT NULL DEFAULT 'Pending',
    zoho_record_id VARCHAR(100),
    source VARCHAR(50) NOT NULL
);
```

8.2 MongoDB Raw Audit Log Model

The document model "RawPayloadLog" stores the original JSON string and request metadata:

- **Id:** MongoDB BsonObjectId representing the document key.
- **PayloadType:** Identifies client source (ShopifyWebhook or ZapierRequest).
- **RawJson:** The exact unmodified UTF-8 HTTP request body string.
- **Headers:** Dictionary of all request headers, preserving signature data.

9. Implementation

The pipeline is written in C# targeting .NET 10.0. Security, validation, database repository access, and Zoho CRM clients coordinate seamlessly using standard dependency injection.

9.1 Shopify HMAC Verification Logic

The verification calculates the SHA256 HMAC of the request body bytes using the secret key, converts it to a Base64 string, and compares it to the header using constant-time comparison:

```
var hmac = new HMACSHA256(Encoding.UTF8.GetBytes(secret));
var computed = Convert.ToBase64String(hmac.ComputeHash(bodyBytes));
return CryptographicOperations.FixedTimeEquals(
    Encoding.UTF8.GetBytes(computed),
    Encoding.UTF8.GetBytes(hmacHeader));
```

9.2 Zoho CRM Outbound Service

ZohoCrmService fetches short-lived OAuth access tokens using the refresh token flow, caching them until expiration. In mock mode (UseMock = true), the service bypasses external API requests, logging formatted payloads to the console and returning simulated record IDs (e.g., ZOHO_MOCK_...) to verify the pipeline.

10. Deployment Context and Infrastructure

The pipeline uses Docker containerization to bundle database dependencies, facilitating one-click deployments. The deployment is defined using two primary assets:

10.1 Dockerfile (ASP.NET Core Web API)

Multi-stage build compiling project in a dotnet 10.0 SDK container and executing inside a lightweight aspnet 10.0 runtime container.

10.2 Docker Compose Configuration

Manages network and volumes for PostgreSQL (16) and MongoDB (7) images, ensuring healthchecks are met before launching the backend container. Ports are mapped to: Web API (5080), Postgres (5432), and MongoDB (27017).

11. Testing and Validation

To verify the architecture, we ran automated test cases covering security, validation, database integration, and CRM updates under mock databases.

ID	Scenario	Input Data	Expected Result	Status
TC001	Valid Webhook	Valid HMAC + JSON	Record accepted, Zoho sync success	Passed
TC002	Invalid HMAC	Bad HMAC header	401 Unauthorized, rejected	Passed
TC003	Missing Fields	JSON missing OrderId	400 Bad Request, validation fail	Passed
TC004	Valid Zapier JWT	Bearer JWT header	Auth success, order processed	Passed
TC005	Duplicate Order	SHOP-777 sent twice	Database filters duplicate, OK returned	Passed

11.1 HMAC verification log proof

Running test-shopify.ps1 prints: "Shopify webhook signature verified successfully" and completes the relational write, showing full ingress security compliance.

12. Conclusion

The project successfully implements a containerized, polyglot persistence pipeline integrating Shopify and Zapier transaction streams with Zoho CRM. All security requirements, including SHA256 HMAC verification and JWT token gating, have been verified. The application is ready for production staging deployment.

13. Challenges and Lessons Learned

13.1 Polyglot Transaction Synchronization

Synchronizing data across relational, document, and CRM APIs simultaneously introduces network latency risk. Executing DB writes asynchronously and implementing mock settings enabled testing without live dependency bottlenecks.

13.2 Shopify HMAC request stream reading

ASP.NET Core consumes request bodies during model binding, blocking signature checks. Enabling Request Buffering resolved this, allowing signature calculations and model binding to read the body stream consecutively.

14. Future Scope

1. Integration with a live Zoho CRM instance using production client credentials.
2. Implementing a Redis cache layer for JWT tokens to support horizontal API scaling.
3. Exposing a frontend dashboard using ReactJS to monitor order volumes and integration success rates in real time.

15. References

[1] Richardson, C. (2018). Microservices Patterns: With examples in Java. Manning Publications.
[2] Newman, S. (2019). Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith. O'Reilly Media.
[3] Sadalage, P. J., & Fowler, M. (2012). NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley.
[4] RFC 7519: JSON Web Token (JWT). Internet Engineering Task Force (IETF).
[5] RFC 2104: HMAC: Keyed-Hashing for Message Authentication. Internet Engineering Task Force (IETF).
[6] Entity Framework Core Core Concepts. Microsoft Documentation.